exploration (trying new things) versus exploitation (taking the best action you know of). In this chapter we used the standard epsilon-greedy approach to select actions, where we initially take random actions to explore, and then progressively switch our strategy to taking the highest value actions.

The Q function must be learned from data. The Q function has to learn how to make accurate Q value predictions of states. The Q function could be anything really—anything from an unintelligent database to a complex deep learning algorithm. Since deep learning is the best class of learning algorithms we have at the moment, we employed neural networks as our Q functions. This means that "learning the Q function" is the same as training a neural network with backpropagation.

One important concept about Q-learning that we held back until now is that it is an *off-policy* algorithm, in contrast to an *on-policy* algorithm. You already know what a policy is from the last chapter: it's the strategy an algorithm uses to maximize rewards over time. If a human is learning to play Gridworld, they might employ a policy that first scouts all possible paths toward the goal and then selects the one that is shortest. Another policy might be to randomly take actions until you land on the goal.

An off-policy reinforcement learning algorithm like Q-learning means that the choice of policy does not affect the ability to learn accurate Q values. Indeed, our Q-network could learn accurate Q values if we selected actions at random; eventually it would experience a number of winning and losing games and infer the values of states and actions. Of course, this is terribly inefficient, but the policy matters only insofar as it helps us learn with the least amount of data. In contrast, an on-policy algorithm will explicitly depend on the choice of policy or will directly aim at learning a policy from the data. In other words, in order to train our DQN, we need to collect data (experiences) from the environment, and we could do this using any policy, so DQN is off-policy. In contrast, an on-policy algorithm learns a policy while simultaneously using the same policy to collect experiences for training itself.

Another key concept we've saved until now is the notion of *model-based* versus *model-free* algorithms. To make sense of this, we first need to